Comprehensive Final Document

Smart Ledger Accounting App for Merchants and Service Providers

Version: 1.0 — Final for Implementation**

Language: English (Translated from Arabic)**

System: Full-stack (Flutter Frontend + NestJS Backend + PostgreSQL)**

Scope: Phase 1 (Yemen) → Phase 2 (Arab World)**

Table of Contents

١. [Vision and Scope](https://www.google.com/search?q=%231-vision-and-scope)

٢. [Architectural Philosophy and Governing Principles](https://www.google.com/search?q=%232-architectural-philosophy-and-governing-principles)

٣. [Approved Tech Stack](https://www.google.com/search?q=%233-approved-tech-stack)

٤. [Full Database Design (DDL)](https://www.google.com/search?q=%234-full-database-design-ddl)

٥. [Role-Based Access Control (RBAC) Model](https://www.google.com/search?q=%235-role-based-access-control-rbac-model)

٦. [App Screens, Buttons, and Operations](https://www.google.com/search?q=%236-app-screens-buttons-and-operations)

٧. [AI Layer](https://www.google.com/search?q=%237-ai-layer)

٨. [Synchronization and Offline Mode](https://www.google.com/search?q=%238-synchronization-and-offline-mode)

٩. [Phased Implementation Plan](https://www.google.com/search?q=%239-phased-implementation-plan)

١٠. [Quality and Security Standards (DoD)](https://www.google.com/search?q=%2310-quality-and-security-standards)

١١. [Appendices: Migrations and Seed Data](https://www.google.com/search?q=%2311-appendices-migrations-and-seed-data)

---

١. Vision and Scope

١,١ Application Description

A full-stack, smart accounting application designed for goods and services merchants, built on solid accounting principles. It records double-entry bookkeeping (debit/credit) behind intuitive, easy-to-understand buttons for non-accountants. It features an AI agent capable of understanding text and voice descriptions in colloquial dialects, supports offline operations with cross-device synchronization for a single account, and is highly scalable up to 5 million users.

١,٢ Target Audience

* Retail merchants (shops, stalls, small warehouses)
* Service providers (electricians, plumbers, consultants, etc.)
* Small and Medium Enterprises (SMEs) in Yemen and the Arab World

١,٣ Core Principles

Principle	Implementation
Simplicity of Use**	Every operation = 1 Button / Simple Wizard. Accounting jargon only appears in an optional ** advanced mode.
Accounting Accuracy**	Every journal entry is forced to balance (SUM Debit = SUM Credit) at the database ** constraint level.
Offline-Capable**	Local-First architecture using local SQLite + eventual synchronization**.
Multi-Tenant Isolation**	Multi-tenancy from day one (every company = an isolated Tenant)**.
Transparency**	AI never posts an entry automatically; explicit user confirmation is always required**.

---

٢. Architectural Philosophy and Governing Principles

٢,١ Overall Design Pattern

Local-First + Offline-First Architecture

* The local database (SQLite via Drift/SQLCipher) is the immediate source of truth.
* Every operation is written locally first, then appended to a `sync_queue` to sync when the internet is available.
* Synchronization uses an `operation_id` (UUID) to ensure idempotency and prevent duplication.
* Conflict Resolution: Last-Write-Wins for metadata, Append-only for financial journal entries.

٢,٢ Backend Pattern

Modular Monolith designed to easily break into Microservices later:**

* `accounting` — The journal entry engine (the core)
* `inventory` — Stock and items
* `contacts` — Customers/Suppliers
* `hr-payroll` — Employees and payroll
* `ai-gateway` — Intermediate AI routing layer
* `reports` — Reporting and exports
* `sync` — Synchronization service
* `notifications` — Alerts (Push/SMS/WhatsApp)

٢,٣ Non-Negotiable Engineering Principles

١. **Double-Entry Integrity:** Every accounting entry MUST have matching debits and credits, enforced via a DB constraint.

٢. **Immutability for Posted Entries:** No direct editing of a posted entry. Corrections are made strictly via a Reversal Entry.

٣. **Row-Level Security (RLS):** Strict isolation of company data (Tenant) at the PostgreSQL level, not just via middleware.

٤. **Human-in-the-Loop:** Any AI suggestion requires explicit user confirmation before saving.

٥. **API-First:** The backend is built as a REST API documented with OpenAPI before building any UI screens.

٦. **UUID Keys:** All tables use UUID v7 (or BIGINT with public_id) to support offline record creation seamlessly.

---

٣. Approved Tech Stack

Layer	Technology	Purpose
Frontend	Flutter 3.x + Riverpod	Unified interface for iOS/Android/Web/Desktop (Native Mobile App Focus)**
Local DB	Drift (SQLite) + SQLCipher	Encrypted local storage, offline functionality**
Backend	NestJS (Node.js) + TypeScript	Structured, highly modular custom backend (No n8n dependency)**
Database	PostgreSQL 15+	Central database (ACID, JSONB, Partitioning)**

| Cache & Queue** | Redis + BullMQ | Session management, task queues, instant notifications** |

| ORM** | Prisma / TypeORM | Schema management and migrations** |

| Auth** | JWT + Refresh Tokens + OTP | Login, two-factor authentication** |

AI Gateway** | Independent Adapter Service | Supports OpenAI, Anthropic, Gemini, Ollama with easy provider ** |
| switching

| Speech-to-Text** | Whisper API / Self-hosted | Voice input support for Arabic/Yemeni dialects** |

| Vector DB** | pgvector (in PostgreSQL) | RAG knowledge base for accounting consultations** |

| Storage** | S3-compatible (MinIO/AWS S3) | Attachments, reports, backups** |

| Infra** | Docker → Kubernetes | Containerization, horizontal scaling when needed** |

| Monitoring** | Prometheus + Grafana + Sentry | Performance and error monitoring** |

| Secrets** | HashiCorp Vault / AWS Secrets Manager | Encrypted API key storage** |

---

٤. Full Database Design (DDL)

All tables below are ready for implementation on **PostgreSQL 15+**. **Row-Level Security (RLS)** is applied to every table containing a `company_id` to strictly isolate tenants.

> *Note: Extensions `pgcrypto`, `pgvector`, and `uuid-ossp` or `pg_uuidv7` must be enabled in the database.*

```sql
-- ============================================
-- Required Extensions
-- ============================================
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
CREATE EXTENSION IF NOT EXISTS "pgcrypto"; -- For encryption if needed
CREATE EXTENSION IF NOT EXISTS "pgvector"; -- For RAG knowledge base

-- ============================================
-- ١. Reference Data
-- ============================================

CREATE TABLE countries (
    code CHAR(2) PRIMARY KEY,
```

```
,name_ar VARCHAR(100) NOT NULL
,name_en VARCHAR(100) NOT NULL
,default_currency CHAR(3) NOT NULL
,phone_prefix VARCHAR(5) NOT NULL
,is_active BOOLEAN NOT NULL DEFAULT true
()created_at TIMESTAMPTZ NOT NULL DEFAULT now
);

INSERT INTO countries (code, name_ar, name_en, default_currency, phone_prefix) VALUES
('YE', 'اليمن', 'Yemen', 'YER', '+967'),
('SA', 'السعودية', 'Saudi Arabia', 'SAR', '+966'),
('EG', 'مصر', 'Egypt', 'EGP', '+20'),
('AE', 'الإمارات', 'UAE', 'AED', '+971');

) CREATE TABLE currencies
,code CHAR(3) PRIMARY KEY
,name_ar VARCHAR(50) NOT NULL
,symbol VARCHAR(10) NOT NULL
,decimal_places SMALLINT NOT NULL DEFAULT 2
is_active BOOLEAN NOT NULL DEFAULT true
);

INSERT INTO currencies (code, name_ar, symbol, decimal_places) VALUES
('YER', 'الريال اليمني', 'YER', 0),
('SAR', 'الريال السعودي', 'SAR', 2),
('EGP', 'الجنيه المصري', 'EGP', 2),
('AED', 'درهم إماراتي', 'AED', 2),
('USD', 'دولار أمريكي', '$', ٢);

) CREATE TABLE exchange_rates
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,from_currency CHAR(3) NOT NULL REFERENCES currencies(code)
,to_currency CHAR(3) NOT NULL REFERENCES currencies(code)
,rate NUMERIC(18,6) NOT NULL
```

```
,effective_date DATE NOT NULL
,()created_at TIMESTAMPTZ NOT NULL DEFAULT now
UNIQUE(from_currency, to_currency, effective_date)
;(

-- ========================================

-- ٢. Identity and Companies (Multi-Tenancy)

-- ========================================

) CREATE TABLE companies
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,name VARCHAR(200) NOT NULL
,activity_type VARCHAR(30) NOT NULL CHECK (activity_type IN ('retail','services','mixed'))
,country_code CHAR(2) NOT NULL REFERENCES countries(code)
,base_currency CHAR(3) NOT NULL REFERENCES currencies(code)
,fiscal_year_start SMALLINT NOT NULL DEFAULT 1
,logo_url TEXT
'status VARCHAR(20) NOT NULL DEFAULT 'active
,CHECK (status IN ('active','suspended','deleted'))
,'{}' settings_json JSONB NOT NULL DEFAULT
,()created_at TIMESTAMPTZ NOT NULL DEFAULT now
()updated_at TIMESTAMPTZ NOT NULL DEFAULT now
;(

;CREATE INDEX idx_companies_status ON companies(status)

) CREATE TABLE branches
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,company_id UUID NOT NULL REFERENCES companies(id) ON DELETE CASCADE
,name VARCHAR(100) NOT NULL
,is_main BOOLEAN NOT NULL DEFAULT false
()created_at TIMESTAMPTZ NOT NULL DEFAULT now
;(

) CREATE TABLE users
```

,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4

,company_id UUID NOT NULL REFERENCES companies(id) ON DELETE CASCADE

,name VARCHAR(100) NOT NULL

,phone VARCHAR(20) NOT NULL

,(١٥٠)email VARCHAR

,pin_hash TEXT

,password_hash TEXT

,biometric_enabled BOOLEAN NOT NULL DEFAULT false

'status VARCHAR(20) NOT NULL DEFAULT 'active

,CHECK (status IN ('active','suspended','invited'))

,last_active_at TIMESTAMPTZ

,()created_at TIMESTAMPTZ NOT NULL DEFAULT now

,()updated_at TIMESTAMPTZ NOT NULL DEFAULT now

,deleted_at TIMESTAMPTZ

UNIQUE(company_id, phone)

;(

;CREATE INDEX idx_users_company ON users(company_id) WHERE deleted_at IS NULL

) CREATE TABLE user_sessions

,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4

,user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE

,device_id UUID

,refresh_token_hash TEXT NOT NULL

,ip_address INET

,user_agent TEXT

,expires_at TIMESTAMPTZ NOT NULL

,()created_at TIMESTAMPTZ NOT NULL DEFAULT now

revoked_at TIMESTAMPTZ

;(

) CREATE TABLE otp_requests

,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4

,phone VARCHAR(20) NOT NULL

```
,code_hash TEXT NOT NULL
,purpose VARCHAR(20) NOT NULL
,attempts SMALLINT NOT NULL DEFAULT 0
,expires_at TIMESTAMPTZ NOT NULL
,verified_at TIMESTAMPTZ
()created_at TIMESTAMPTZ NOT NULL DEFAULT now
;(

;CREATE INDEX idx_otp_phone_expires ON otp_requests(phone, expires_at)

-- ========================================

-- ٣. RBAC Model - Tenant Level

-- ========================================

) CREATE TABLE permissions
,code VARCHAR(100) PRIMARY KEY
,module VARCHAR(50) NOT NULL
,label_ar VARCHAR(150) NOT NULL
,description_ar TEXT
is_sensitive BOOLEAN NOT NULL DEFAULT false
;(

) CREATE TABLE roles
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
company_id UUID REFERENCES companies(id) ON DELETE CASCADE, -- NULL = System default role
,name VARCHAR(50) NOT NULL
,is_system BOOLEAN NOT NULL DEFAULT false
()created_at TIMESTAMPTZ NOT NULL DEFAULT now
;(

) CREATE TABLE role_permissions
,role_id UUID NOT NULL REFERENCES roles(id) ON DELETE CASCADE
,permission_code VARCHAR(100) NOT NULL REFERENCES permissions(code)
,is_allowed BOOLEAN NOT NULL DEFAULT true
PRIMARY KEY(role_id, permission_code)
```

```
);

) CREATE TABLE user_roles
,user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE
,role_id UUID NOT NULL REFERENCES roles(id)
,company_id UUID NOT NULL REFERENCES companies(id)
,()assigned_at TIMESTAMPTZ NOT NULL DEFAULT now
,assigned_by UUID REFERENCES users(id)
PRIMARY KEY(user_id, role_id)
);

;CREATE INDEX idx_user_roles_company ON user_roles(company_id)

-- ========================================== --
-- ٤. Core Accounting Engine --
-- ========================================== --

) CREATE TABLE account_types
,code VARCHAR(20) PRIMARY KEY
,name_ar VARCHAR(50) NOT NULL
normal_balance VARCHAR(10) NOT NULL CHECK (normal_balance IN ('debit','credit'))
);

INSERT INTO account_types (code, name_ar, normal_balance) VALUES
('asset', 'أصول', 'debit'),
('liability', 'خصوم', 'credit'),
('equity', 'حقوق ملكية', 'credit'),
('revenue', 'إيرادات', 'credit'),
('expense', 'مصروفات', 'debit');

) CREATE TABLE accounts
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,company_id UUID NOT NULL REFERENCES companies(id) ON DELETE CASCADE
,code VARCHAR(20) NOT NULL
,name_ar VARCHAR(150) NOT NULL
```

```
,account_type VARCHAR(20) NOT NULL REFERENCES account_types(code)
,parent_id UUID REFERENCES accounts(id)
... ,account_role VARCHAR(50), -- cash, bank, sales_revenue
,is_system_generated BOOLEAN NOT NULL DEFAULT false
,is_active BOOLEAN NOT NULL DEFAULT true
,currency CHAR(3) REFERENCES currencies(code)
,cached_balance NUMERIC(18,4) NOT NULL DEFAULT 0
,()created_at TIMESTAMPTZ NOT NULL DEFAULT now
,()updated_at TIMESTAMPTZ NOT NULL DEFAULT now
UNIQUE(company_id, code)
;(

;CREATE INDEX idx_accounts_company_active ON accounts(company_id) WHERE is_active = true
;CREATE INDEX idx_accounts_role ON accounts(company_id, account_role)

) CREATE TABLE posting_rules
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
company_id UUID REFERENCES companies(id) ON DELETE CASCADE, -- NULL = System
,action_code VARCHAR(50) NOT NULL
,label_ar VARCHAR(100) NOT NULL
,description_ui VARCHAR(255) NOT NULL
,category VARCHAR(30) NOT NULL
,(°ㅇ)icon VARCHAR
,template_json JSONB NOT NULL
,requires_approval BOOLEAN NOT NULL DEFAULT false
,is_active BOOLEAN NOT NULL DEFAULT true
,display_order SMALLINT NOT NULL DEFAULT 0
()created_at TIMESTAMPTZ NOT NULL DEFAULT now
;(

) CREATE TABLE fiscal_periods
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,company_id UUID NOT NULL REFERENCES companies(id) ON DELETE CASCADE
,period_start DATE NOT NULL
```

```
,period_end DATE NOT NULL
,status VARCHAR(20) NOT NULL DEFAULT 'open' CHECK (status IN ('open','closed','locked'))
,closed_by UUID REFERENCES users(id)
,closed_at TIMESTAMPTZ
UNIQUE(company_id, period_start, period_end)
;(

-- Journal Entries (Header) - Partitioned Monthly
) CREATE TABLE journal_entries
,()id UUID NOT NULL DEFAULT uuid_generate_v4
,company_id UUID NOT NULL REFERENCES companies(id) ON DELETE CASCADE
,entry_number BIGINT NOT NULL
,entry_date DATE NOT NULL
,fiscal_period_id UUID REFERENCES fiscal_periods(id)
,reference_type VARCHAR(30) NOT NULL
,reference_id UUID
,posting_rule_id UUID REFERENCES posting_rules(id)
,description TEXT NOT NULL
'source VARCHAR(20) NOT NULL DEFAULT 'manual
,CHECK (source IN ('manual','button','ai','system'))
'status VARCHAR(20) NOT NULL DEFAULT 'posted
,CHECK (status IN ('draft','posted','reversed'))
,reversal_of UUID
,created_by UUID REFERENCES users(id)
,operation_id UUID NOT NULL
,device_id UUID
,()created_at TIMESTAMPTZ NOT NULL DEFAULT now
PRIMARY KEY (id, entry_date)
;(PARTITION BY RANGE (entry_date )

-- Trial partition for January 2025 (Automated via pg_partman or cron job)
CREATE TABLE journal_entries_2025_01 PARTITION OF journal_entries
;('٠١-٠٢-٢٠٢٥') FOR VALUES FROM ('2025-01-01') TO
```

```
;CREATE UNIQUE INDEX idx_journal_operation_id ON journal_entries(operation_id)
;CREATE INDEX idx_journal_company_date ON journal_entries(company_id, entry_date DESC)
;CREATE INDEX idx_journal_reference ON journal_entries(reference_type, reference_id)
;CREATE UNIQUE INDEX idx_journal_number ON journal_entries(company_id, entry_number)

) CREATE TABLE journal_entry_lines
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,journal_entry_id UUID NOT NULL
,entry_date DATE NOT NULL
,account_id UUID NOT NULL REFERENCES accounts(id)
,debit NUMERIC(18,4) NOT NULL DEFAULT 0 CHECK (debit >= 0)
,credit NUMERIC(18,4) NOT NULL DEFAULT 0 CHECK (credit >= 0)
,contact_id UUID REFERENCES contacts(id)
,(٢٥٥)memo VARCHAR
,line_order SMALLINT NOT NULL DEFAULT 0
) CONSTRAINT chk_single_side CHECK
(debit > 0 AND credit = 0) OR (credit > 0 AND debit = 0)
,(
FOREIGN KEY (journal_entry_id, entry_date) REFERENCES journal_entries(id, entry_date)
;(

;CREATE INDEX idx_jel_entry ON journal_entry_lines(journal_entry_id)
;CREATE INDEX idx_jel_account ON journal_entry_lines(account_id)
;CREATE INDEX idx_jel_contact ON journal_entry_lines(contact_id) WHERE contact_id IS NOT NULL

-- Trigger: Enforce Double-Entry Balance before Commit
$$ CREATE OR REPLACE FUNCTION fn_validate_journal_balance() RETURNS TRIGGER AS
DECLARE
;(١٨,٤)total_debit NUMERIC
;(١٨,٤)total_credit NUMERIC
BEGIN
SELECT COALESCE(SUM(debit),0), COALESCE(SUM(credit),0)
INTO total_debit, total_credit
FROM journal_entry_lines
```

```
;WHERE journal_entry_id = NEW.journal_entry_id

IF total_debit <> total_credit THEN
;RAISE EXCEPTION 'Unbalanced Entry: Debit % != Credit %', total_debit, total_credit
;END IF
;RETURN NEW
;END
;LANGUAGE plpgsql $$

CREATE CONSTRAINT TRIGGER trg_validate_balance
AFTER INSERT OR UPDATE ON journal_entry_lines
DEFERRABLE INITIALLY DEFERRED
;()FOR EACH ROW EXECUTE FUNCTION fn_validate_journal_balance

Trigger: Prevent editing of a posted entry --
$$ CREATE OR REPLACE FUNCTION fn_prevent_posted_edit() RETURNS TRIGGER AS
BEGIN
IF OLD.status = 'posted' THEN
;'.RAISE EXCEPTION 'Cannot edit a posted entry. Use a reversal entry instead
;END IF
;RETURN NEW
;END
;LANGUAGE plpgsql $$

CREATE TRIGGER trg_prevent_edit
BEFORE UPDATE OR DELETE ON journal_entries
;()FOR EACH ROW EXECUTE FUNCTION fn_prevent_posted_edit

========================================== --
Contacts / Sub-Ledgers .° --
========================================== --

) CREATE TABLE contacts
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
```

```
,company_id UUID NOT NULL REFERENCES companies(id) ON DELETE CASCADE
,type VARCHAR(20) NOT NULL CHECK (type IN ('customer','supplier','both'))
,name VARCHAR(150) NOT NULL
,(٢٠)phone VARCHAR
,(١٥٠)email VARCHAR
,address TEXT
,(٥٠)tax_number VARCHAR
,(١٨,٤)credit_limit NUMERIC
,payment_terms_days SMALLINT DEFAULT 0
,cached_balance NUMERIC(18,4) NOT NULL DEFAULT 0
,is_active BOOLEAN NOT NULL DEFAULT true
,()created_at TIMESTAMPTZ NOT NULL DEFAULT now
,()updated_at TIMESTAMPTZ NOT NULL DEFAULT now
deleted_at TIMESTAMPTZ
;(

;CREATE INDEX idx_contacts_company ON contacts(company_id) WHERE deleted_at IS NULL
;CREATE INDEX idx_contacts_balance ON contacts(company_id, cached_balance) WHERE cached_balance <> 0

-- ========================================== --
-- ٦. Sales, Purchases, and Expenses --
-- ========================================== --

) CREATE TABLE sales_invoices
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,company_id UUID NOT NULL REFERENCES companies(id) ON DELETE CASCADE
,invoice_number BIGINT NOT NULL
,contact_id UUID REFERENCES contacts(id)
,journal_entry_id UUID
,invoice_date DATE NOT NULL
,payment_type VARCHAR(10) NOT NULL CHECK (payment_type IN ('cash','credit'))
,subtotal NUMERIC(18,4) NOT NULL
,discount NUMERIC(18,4) NOT NULL DEFAULT 0
,tax_amount NUMERIC(18,4) NOT NULL DEFAULT 0
```

```
,total NUMERIC(18,4) NOT NULL
'status VARCHAR(20) NOT NULL DEFAULT 'confirmed
,CHECK (status IN ('draft','confirmed','voided'))
,operation_id UUID NOT NULL
,created_by UUID REFERENCES users(id)
,()created_at TIMESTAMPTZ NOT NULL DEFAULT now
UNIQUE(company_id, invoice_number)
;(

;CREATE UNIQUE INDEX idx_sales_operation ON sales_invoices(operation_id)
;CREATE INDEX idx_sales_company_date ON sales_invoices(company_id, invoice_date DESC)

) CREATE TABLE sales_invoice_items
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,sales_invoice_id UUID NOT NULL REFERENCES sales_invoices(id) ON DELETE CASCADE
,item_id UUID REFERENCES items(id)
,(٢٥٥)description VARCHAR
,quantity NUMERIC(14,3) NOT NULL
,unit_price NUMERIC(18,4) NOT NULL
,(١٨,٤)cost_price NUMERIC
line_total NUMERIC(18,4) NOT NULL
;(

) CREATE TABLE purchase_invoices
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,company_id UUID NOT NULL REFERENCES companies(id) ON DELETE CASCADE
,invoice_number BIGINT NOT NULL
,contact_id UUID REFERENCES contacts(id)
,journal_entry_id UUID
,invoice_date DATE NOT NULL
,payment_type VARCHAR(10) NOT NULL CHECK (payment_type IN ('cash','credit'))
,total NUMERIC(18,4) NOT NULL
,'status VARCHAR(20) NOT NULL DEFAULT 'confirmed
,operation_id UUID NOT NULL
```

```
,created_by UUID REFERENCES users(id)
,()created_at TIMESTAMPTZ NOT NULL DEFAULT now
UNIQUE(company_id, invoice_number)
;(

) CREATE TABLE purchase_invoice_items
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,purchase_invoice_id UUID NOT NULL REFERENCES purchase_invoices(id) ON DELETE CASCADE
,item_id UUID REFERENCES items(id)
,quantity NUMERIC(14,3) NOT NULL
,unit_cost NUMERIC(18,4) NOT NULL
line_total NUMERIC(18,4) NOT NULL
;(

) CREATE TABLE expense_categories
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,company_id UUID REFERENCES companies(id) ON DELETE CASCADE
,name_ar VARCHAR(100) NOT NULL
default_account_id UUID REFERENCES accounts(id)
;(

) CREATE TABLE expense_records
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,company_id UUID NOT NULL REFERENCES companies(id) ON DELETE CASCADE
,category_id UUID REFERENCES expense_categories(id)
,journal_entry_id UUID
,amount NUMERIC(18,4) NOT NULL
,(٢٥٥)description VARCHAR
,expense_date DATE NOT NULL
,requires_approval BOOLEAN NOT NULL DEFAULT false
,approved_by UUID REFERENCES users(id)
,approved_at TIMESTAMPTZ
,attachment_id UUID REFERENCES attachments(id)
,operation_id UUID NOT NULL
```

```
,created_by UUID REFERENCES users(id)
()created_at TIMESTAMPTZ NOT NULL DEFAULT now
;(

;CREATE INDEX idx_expenses_company_date ON expense_records(company_id, expense_date DESC)

-- ========================================

-- ٧. Inventory (Perpetual System)

-- ========================================

) CREATE TABLE item_categories
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,company_id UUID NOT NULL REFERENCES companies(id) ON DELETE CASCADE
,name_ar VARCHAR(100) NOT NULL
parent_id UUID REFERENCES item_categories(id)
;(

) CREATE TABLE items
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,company_id UUID NOT NULL REFERENCES companies(id) ON DELETE CASCADE
,category_id UUID REFERENCES item_categories(id)
,(٥٠)sku VARCHAR
,(٥٠)barcode VARCHAR
,name VARCHAR(150) NOT NULL
,'unit VARCHAR(20) NOT NULL DEFAULT 'piece
,cost_price NUMERIC(18,4) NOT NULL DEFAULT 0
,sell_price NUMERIC(18,4) NOT NULL DEFAULT 0
,quantity_on_hand NUMERIC(14,3) NOT NULL DEFAULT 0
,reorder_level NUMERIC(14,3) DEFAULT 0
,expiry_date DATE
,is_service BOOLEAN NOT NULL DEFAULT false
,is_active BOOLEAN NOT NULL DEFAULT true
,image_url TEXT
,()created_at TIMESTAMPTZ NOT NULL DEFAULT now
```

```
,()updated_at TIMESTAMPTZ NOT NULL DEFAULT now
,deleted_at TIMESTAMPTZ
UNIQUE(company_id, sku)
;(

;CREATE INDEX idx_items_company ON items(company_id) WHERE deleted_at IS NULL
;CREATE INDEX idx_items_barcode ON items(company_id, barcode)
;CREATE INDEX idx_items_low_stock ON items(company_id) WHERE quantity_on_hand <= reorder_level
;CREATE INDEX idx_items_expiry ON items(company_id, expiry_date) WHERE expiry_date IS NOT NULL

) CREATE TABLE stock_movements
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,company_id UUID NOT NULL REFERENCES companies(id) ON DELETE CASCADE
,item_id UUID NOT NULL REFERENCES items(id)
movement_type VARCHAR(20) NOT NULL CHECK (movement_type IN
,('purchase_in','sale_out','adjustment','damage','return_in','return_out'))
,quantity NUMERIC(14,3) NOT NULL
,(٣٠)reference_type VARCHAR
,reference_id UUID
,balance_after NUMERIC(14,3) NOT NULL
,notes TEXT
,created_by UUID REFERENCES users(id)
()created_at TIMESTAMPTZ NOT NULL DEFAULT now
;(

;CREATE INDEX idx_stock_movements_item ON stock_movements(item_id, created_at DESC)

-- ========================================

-- ٨. Human Resources & Fixed Assets

-- ========================================

) CREATE TABLE employees
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,company_id UUID NOT NULL REFERENCES companies(id) ON DELETE CASCADE
,user_id UUID REFERENCES users(id)
```

```
,name VARCHAR(150) NOT NULL
,(١٠٠)position VARCHAR
,base_salary NUMERIC(18,4) NOT NULL DEFAULT 0
,hire_date DATE
,status VARCHAR(20) NOT NULL DEFAULT 'active' CHECK (status IN ('active','terminated'))
,()created_at TIMESTAMPTZ NOT NULL DEFAULT now
deleted_at TIMESTAMPTZ
;(

) CREATE TABLE payroll_runs
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,company_id UUID NOT NULL REFERENCES companies(id) ON DELETE CASCADE
,period_month SMALLINT NOT NULL
,period_year SMALLINT NOT NULL
,status VARCHAR(20) NOT NULL DEFAULT 'draft' CHECK (status IN ('draft','paid'))
,(١٨,٤)total_amount NUMERIC
,journal_entry_id UUID
,()created_at TIMESTAMPTZ NOT NULL DEFAULT now
UNIQUE(company_id, period_month, period_year)
;(

) CREATE TABLE payroll_items
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,payroll_run_id UUID NOT NULL REFERENCES payroll_runs(id) ON DELETE CASCADE
,employee_id UUID NOT NULL REFERENCES employees(id)
,base_salary NUMERIC(18,4) NOT NULL
,deductions NUMERIC(18,4) NOT NULL DEFAULT 0
,bonuses NUMERIC(18,4) NOT NULL DEFAULT 0
net_amount NUMERIC(18,4) NOT NULL
;(

) CREATE TABLE salary_advances
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,employee_id UUID NOT NULL REFERENCES employees(id)
```

```
,amount NUMERIC(18,4) NOT NULL
,advance_date DATE NOT NULL
,journal_entry_id UUID
settled BOOLEAN NOT NULL DEFAULT false
;(

) CREATE TABLE fixed_assets
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,company_id UUID NOT NULL REFERENCES companies(id) ON DELETE CASCADE
,name VARCHAR(150) NOT NULL
,cost NUMERIC(18,4) NOT NULL
,acquired_date DATE NOT NULL
,'depreciation_method VARCHAR(20) NOT NULL DEFAULT 'straight_line
,useful_life_months SMALLINT NOT NULL
,salvage_value NUMERIC(18,4) NOT NULL DEFAULT 0
,accumulated_depreciation NUMERIC(18,4) NOT NULL DEFAULT 0
,status VARCHAR(20) NOT NULL DEFAULT 'active' CHECK (status IN ('active','disposed'))
()created_at TIMESTAMPTZ NOT NULL DEFAULT now
;(

) CREATE TABLE depreciation_schedules
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,fixed_asset_id UUID NOT NULL REFERENCES fixed_assets(id) ON DELETE CASCADE
,period_date DATE NOT NULL
,amount NUMERIC(18,4) NOT NULL
,journal_entry_id UUID
()created_at TIMESTAMPTZ NOT NULL DEFAULT now
;(

========================================== --
AI Layer .٩ --
========================================== --

) CREATE TABLE ai_providers
```

```
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,company_id UUID REFERENCES companies(id) ON DELETE CASCADE
,provider_name VARCHAR(50) NOT NULL
,model_name VARCHAR(100) NOT NULL
,encrypted_api_key TEXT
,is_default BOOLEAN NOT NULL DEFAULT false
,is_active BOOLEAN NOT NULL DEFAULT true
,monthly_token_limit BIGINT
()created_at TIMESTAMPTZ NOT NULL DEFAULT now
;(

) CREATE TABLE ai_conversations
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,company_id UUID NOT NULL REFERENCES companies(id) ON DELETE CASCADE
,user_id UUID NOT NULL REFERENCES users(id)
,channel VARCHAR(10) NOT NULL CHECK (channel IN ('text','voice'))
,()started_at TIMESTAMPTZ NOT NULL DEFAULT now
last_message_at TIMESTAMPTZ
;(

) CREATE TABLE ai_messages
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,conversation_id UUID NOT NULL REFERENCES ai_conversations(id) ON DELETE CASCADE
,role VARCHAR(10) NOT NULL CHECK (role IN ('user','agent','system'))
,content TEXT NOT NULL
,(٥٠)intent_detected VARCHAR
,metadata_json JSONB
()created_at TIMESTAMPTZ NOT NULL DEFAULT now
;(PARTITION BY RANGE (created_at )

) CREATE TABLE ai_usage_logs
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,company_id UUID NOT NULL REFERENCES companies(id) ON DELETE CASCADE
,provider_id UUID REFERENCES ai_providers(id)
```

```
,user_id UUID REFERENCES users(id)
,tokens_input INTEGER NOT NULL DEFAULT 0
,tokens_output INTEGER NOT NULL DEFAULT 0
,(١٠,٤)cost_estimated NUMERIC
,(٣٠)request_type VARCHAR
()created_at TIMESTAMPTZ NOT NULL DEFAULT now
;(PARTITION BY RANGE (created_at )

) CREATE TABLE knowledge_base_documents
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,title VARCHAR(255) NOT NULL
,content TEXT NOT NULL
,(١٥٣٦)embedding VECTOR
,'language VARCHAR(5) DEFAULT 'ar
()created_at TIMESTAMPTZ NOT NULL DEFAULT now
;(

CREATE INDEX idx_kb_embedding ON knowledge_base_documents
;USING ivfflat (embedding vector_cosine_ops)

-- ========================================
-- ١٠. Synchronization & Devices
-- ========================================

) CREATE TABLE devices
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE
,(١٠٠)device_name VARCHAR
,(٢٠)platform VARCHAR
,push_token TEXT
,last_sync_at TIMESTAMPTZ
,(٢٠)app_version VARCHAR
()created_at TIMESTAMPTZ NOT NULL DEFAULT now
;(
```

```
) CREATE TABLE sync_queue
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,device_id UUID NOT NULL REFERENCES devices(id) ON DELETE CASCADE
,company_id UUID NOT NULL REFERENCES companies(id) ON DELETE CASCADE
,entity_type VARCHAR(50) NOT NULL
,entity_id UUID NOT NULL
,operation VARCHAR(10) NOT NULL CHECK (operation IN ('create','update','delete'))
,payload_json JSONB NOT NULL
,operation_id UUID NOT NULL UNIQUE
'status VARCHAR(20) NOT NULL DEFAULT 'pending
,CHECK (status IN ('pending','applied','conflict','failed'))
,error_message TEXT
,()created_at TIMESTAMPTZ NOT NULL DEFAULT now
synced_at TIMESTAMPTZ
;(

;'CREATE INDEX idx_sync_pending ON sync_queue(status, created_at) WHERE status = 'pending

) CREATE TABLE sync_conflicts
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,entity_type VARCHAR(50) NOT NULL
,entity_id UUID NOT NULL
,local_version JSONB NOT NULL
,server_version JSONB NOT NULL
,(٢٠)resolution VARCHAR
,resolved_by UUID REFERENCES users(id)
,resolved_at TIMESTAMPTZ
()created_at TIMESTAMPTZ NOT NULL DEFAULT now
;(

-- ========================================== --
-- ١١. Audit, Attachments, and Notifications --
-- ========================================== --
```

```
) CREATE TABLE audit_logs
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,company_id UUID NOT NULL
,user_id UUID
,action VARCHAR(50) NOT NULL
,entity_type VARCHAR(50) NOT NULL
,entity_id UUID
,old_value JSONB
,new_value JSONB
,ip_address INET
()created_at TIMESTAMPTZ NOT NULL DEFAULT now
;(PARTITION BY RANGE (created_at )

) CREATE TABLE attachments
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,company_id UUID NOT NULL REFERENCES companies(id) ON DELETE CASCADE
,entity_type VARCHAR(50) NOT NULL
,entity_id UUID NOT NULL
,file_url TEXT NOT NULL
,(٢٠)file_type VARCHAR
,file_size_kb INTEGER
,uploaded_by UUID REFERENCES users(id)
()created_at TIMESTAMPTZ NOT NULL DEFAULT now
;(

;CREATE INDEX idx_attachments_entity ON attachments(entity_type, entity_id)

) CREATE TABLE notification_templates
,code VARCHAR(50) PRIMARY KEY
,channel VARCHAR(20) NOT NULL
,title_ar TEXT
body_template_ar TEXT NOT NULL
;(
```

```
) CREATE TABLE notifications
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,company_id UUID NOT NULL REFERENCES companies(id) ON DELETE CASCADE
,user_id UUID REFERENCES users(id)
,template_code VARCHAR(50) REFERENCES notification_templates(code)
,title TEXT
,body TEXT
,channel VARCHAR(20) NOT NULL
'status VARCHAR(20) NOT NULL DEFAULT 'pending
,CHECK (status IN ('pending','sent','failed','read'))
,sent_at TIMESTAMPTZ
,read_at TIMESTAMPTZ
()created_at TIMESTAMPTZ NOT NULL DEFAULT now
;(

;CREATE INDEX idx_notifications_user ON notifications(user_id, status)

========================================== --
Platform Admin .١٢ --
========================================== --

) CREATE TABLE platform_admins
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,name VARCHAR(100) NOT NULL
,email VARCHAR(150) UNIQUE NOT NULL
,password_hash TEXT NOT NULL
,mfa_secret TEXT
,mfa_enabled BOOLEAN NOT NULL DEFAULT false
,'status VARCHAR(20) NOT NULL DEFAULT 'active
()created_at TIMESTAMPTZ NOT NULL DEFAULT now
;(

) CREATE TABLE platform_roles
```

```
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
name VARCHAR(50) NOT NULL UNIQUE
;(

) CREATE TABLE platform_role_permissions
,role_id UUID NOT NULL REFERENCES platform_roles(id) ON DELETE CASCADE
,permission_code VARCHAR(100) NOT NULL
PRIMARY KEY(role_id, permission_code)
;(

) CREATE TABLE platform_admin_roles
,admin_id UUID NOT NULL REFERENCES platform_admins(id) ON DELETE CASCADE
,role_id UUID NOT NULL REFERENCES platform_roles(id) ON DELETE CASCADE
PRIMARY KEY(admin_id, role_id)
;(

) CREATE TABLE support_sessions
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,admin_id UUID NOT NULL REFERENCES platform_admins(id)
,company_id UUID NOT NULL REFERENCES companies(id) ON DELETE CASCADE
,reason TEXT NOT NULL
'consent_status VARCHAR(20) NOT NULL DEFAULT 'pending
,CHECK (consent_status IN ('pending','approved','denied','expired'))
,consent_by_user_id UUID REFERENCES users(id)
,started_at TIMESTAMPTZ
,expires_at TIMESTAMPTZ
,ended_at TIMESTAMPTZ
,'[]' actions_log JSONB NOT NULL DEFAULT
()created_at TIMESTAMPTZ NOT NULL DEFAULT now
;(

) CREATE TABLE support_tickets
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,company_id UUID NOT NULL REFERENCES companies(id) ON DELETE CASCADE
```

```
,user_id UUID REFERENCES users(id)
,assigned_admin_id UUID REFERENCES platform_admins(id)
,subject VARCHAR(255) NOT NULL
'status VARCHAR(20) NOT NULL DEFAULT 'open
,CHECK (status IN ('open','in_progress','resolved','closed'))
,'priority VARCHAR(10) NOT NULL DEFAULT 'normal
()created_at TIMESTAMPTZ NOT NULL DEFAULT now
;(

) CREATE TABLE support_ticket_messages
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,ticket_id UUID NOT NULL REFERENCES support_tickets(id) ON DELETE CASCADE
,sender_type VARCHAR(10) NOT NULL CHECK (sender_type IN ('user','admin'))
,sender_id UUID NOT NULL
,message TEXT NOT NULL
()created_at TIMESTAMPTZ NOT NULL DEFAULT now
;(

) CREATE TABLE subscription_plans
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,name VARCHAR(50) NOT NULL
,price_monthly NUMERIC(10,2) NOT NULL
,'currency CHAR(3) NOT NULL DEFAULT 'USD
,max_users INTEGER
,max_ai_tokens_monthly BIGINT
,'{}' features_json JSONB NOT NULL DEFAULT
is_active BOOLEAN NOT NULL DEFAULT true
;(

) CREATE TABLE subscriptions
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,company_id UUID NOT NULL REFERENCES companies(id) ON DELETE CASCADE UNIQUE
,plan_id UUID NOT NULL REFERENCES subscription_plans(id)
,status VARCHAR(20) NOT NULL DEFAULT 'trial' CHECK (status IN ('trial','active','past_due','canceled'))
```

```
,trial_ends_at TIMESTAMPTZ
,current_period_start TIMESTAMPTZ
,current_period_end TIMESTAMPTZ
()created_at TIMESTAMPTZ NOT NULL DEFAULT now
;(

) CREATE TABLE invoices
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,subscription_id UUID NOT NULL REFERENCES subscriptions(id) ON DELETE CASCADE
,amount NUMERIC(10,2) NOT NULL
,status VARCHAR(20) NOT NULL DEFAULT 'pending' CHECK (status IN ('pending','paid','failed','refunded'))
,(٣٠)payment_method VARCHAR
,()issued_at TIMESTAMPTZ NOT NULL DEFAULT now
paid_at TIMESTAMPTZ
;(

) CREATE TABLE feature_flags
,()id UUID PRIMARY KEY DEFAULT uuid_generate_v4
,flag_key VARCHAR(100) UNIQUE NOT NULL
,description TEXT
,is_enabled_globally BOOLEAN NOT NULL DEFAULT false
,'{}' enabled_for_companies UUID[] DEFAULT
()created_at TIMESTAMPTZ NOT NULL DEFAULT now
;(

========================================== --
Enable Row-Level Security (Tenant Isolation) .١٣ --
========================================== --

;ALTER TABLE journal_entries ENABLE ROW LEVEL SECURITY
;ALTER TABLE journal_entry_lines ENABLE ROW LEVEL SECURITY
;ALTER TABLE accounts ENABLE ROW LEVEL SECURITY
;ALTER TABLE contacts ENABLE ROW LEVEL SECURITY
;ALTER TABLE sales_invoices ENABLE ROW LEVEL SECURITY
```

;ALTER TABLE purchase_invoices ENABLE ROW LEVEL SECURITY

;ALTER TABLE expense_records ENABLE ROW LEVEL SECURITY

;ALTER TABLE items ENABLE ROW LEVEL SECURITY

CREATE POLICY tenant_isolation ON journal_entries

;USING (company_id = current_setting('app.current_tenant')::UUID)

CREATE POLICY tenant_isolation_jel ON journal_entry_lines

USING (EXISTS (SELECT 1 FROM journal_entries je WHERE je.id = journal_entry_id AND je.company_id = ;current_setting('app.current_tenant')::UUID))

The same logic applies to the rest of the tables via an automated script or in the Middleware --

```

---

## ٥. Role-Based Access Control (RBAC) Model

### ٥,١ Roles within the Merchant App (Tenant-Level)

| Role | Core Permissions | Restrictions |

| --- | --- | --- |

| **Owner** | Everything, including deleting the company, managing subscriptions, and inviting users. | Cannot be deleted from the system. |

| **Accountant.** | Journal entries, financial reports, chart of accounts, auditing. | Cannot delete the company** |

| **Sales Staff.** | Register sales, read-only inventory view. | Cannot view financial reports** |

| **Inventory Staff.** | Manage items, stock adjustments, inventory count. | Cannot execute direct sales** |

| **Cashier.** | Cash collection/payments only, cannot edit posted entries. | Subject to configurable amount limits** |

| **Viewer.** | Read-only across the board. | No modification rights** |

| **Custom Role.** | Built by the Owner from the permissions matrix. | Flexible** |

### ٥,٢ Partial Permissions Matrix (Example)

| Permission | Owner | Accountant | Sales | Inventory | Cashier | Viewer |

| --- | --- | --- | --- | --- | --- | --- |

| `sales.create` | ✅ | ✅ | ✅ | ❌ | ❌ | ❌ |

| `sales.view_profit_margin` | ✅ | ✅ | ❌ | ❌ | ❌ | ❌ |
```

| ❌ | ❌ | ❌ | ❌ | ✅ | ✅ | `sales.void` |

| ❌ | expenses.create` | ☑ | ☑ | ❌ | ❌ | ☑ (Limited)` |

| ❌ | ❌ | ❌ | ❌ | ✅ | ✅ | `journal.create_manual` |

| ❌ | ❌ | ❌ | ❌ | ❌ | ❌ | `journal.edit_posted` |

| ❌ | ❌ | ❌ | ❌ | ✅ | ✅ | `hr.process_payroll` |

| ✅ | ❌ | ❌ | ❌ | ✅ | ✅ | `reports.view_financial` |

| ❌ | ❌ | ❌ | ❌ | ❌ | ✅ | `ai.manage_providers_keys` |

| ❌ | ❌ | ❌ | ❌ | ❌ | ✅ | `users.invite` |

| ❌ | ❌ | ❌ | ❌ | ❌ | ✅ | `company.delete` |

Strict Rule:** `journal.edit_posted` is automatically granted to one role only: "System", executed strictly via a ** <
.reversal entry, never to any human user

٥٫٣ Platform Admin Dashboard

| Platform Team Role | Permissions |

| --- | --- |

| .Super Admin | Full access, team role management, tenant deletion |

| .Support Agent | Technical support, Support Mode access (requires merchant consent) |

| .Billing Admin | Plans, billing, and subscriptions |

| .AI Ops Admin | AI providers, consumption monitoring, token limits |

| .Content Manager | Guidance content, RAG knowledge base management |

| .Security Officer | Audit logs, security policies |

| .Analyst | Platform reporting only, read-only |

---

٦. App Screens, Buttons, and Operations

٦٫١ Main Screens (Merchant Native App)

```text

١. Onboarding & Auth

- Login (Phone/OTP), Activity Type Selection (Retail/Services)
```

٢. Dashboard

- Quick summary: Today's sales, expenses, net cash
- Alerts: Overdue debts, low stock
- Floating action button for the Smart Assistant (Voice/Text)

٣. Quick Actions Grid (Main Buttons)

- Cash Sale | Credit Sale | Collect from Customer | Sales Return
- Cash Purchase | Credit Purchase | Pay Supplier | Purchase Return
- General Expense | Payroll | Capital Deposit | Owner's Drawing
- Stock Adjustment | Damaged/Waste | Add Fixed Asset

٤. AI Chat (Smart Conversation)

- Text/Voice, proposes a journal entry with a preview before saving
- ...Queries: Monthly sales, debts, low stock

٥. Contacts Ledger (Sub-ledgers)

- Customer/Supplier account statement, send statement via WhatsApp/Email

٦. Inventory

- Items, quantities, expiry/out-of-stock alerts, barcode scanner

٧. HR & Payroll

- Employees, salaries, advances

٨. Reports

- Income Statement, Balance Sheet, Cash Flow, PDF/Excel export

٩. Settings

- Company, Currency, Taxes, Users, and Permissions
- AI Settings (Provider, Model, Encrypted Key)
- Synchronization and linked devices

```
```

٦٫٢ Accounting Buttons (Posting Rules)

:Every button executes a pre-configured double-entry (Debit/Credit) presented to the user in simple language

| Button | Simplified Description | Debit | Credit |

| --- | --- | --- | --- |

| Cash Sale | Received money for a sale | Cash/Bank | Sales Revenue |

| Credit Sale | Sold on credit | Customers (AR) | Sales Revenue |

| Cash Purchase | Bought and paid immediately | Inventory | Cash |

| Credit Purchase | Bought on credit from a supplier | Inventory | Suppliers (AP) |

| General Expense | Paid rent/electricity/... | Specific Expense | Cash |

| Payroll | Paid employee salaries | Salary Expense | Cash |

| Capital Deposit | Added personal money to the shop | Cash | Capital |

| Owner's Drawing | Withdrew money for personal use | Owner's Drawings | Cash |

Design:** Buttons can be hidden, rearranged, or added from the "Operations Panel Settings" based on the ** <
.activity type

---

٧. AI Layer

٧٫١ AI Gateway Components

* ** .NLU Engine:** Converts Text/Voice into Intents + Entities (Amount, Party, Date)

* ** .Posting Agent:** Proposes a Draft Journal Entry — explicit user confirmation required

* ** .Analytics Agent:** Answers data queries (sales, growth, debts)

* ** .Advisor Agent:** Provides accounting/financial consultation via RAG over the knowledge base + user data

* ** .Onboarding Agent:** Guides the user step-by-step on how to use the app

٧٫٢ Mandatory Safeguards

* ** Function Calling:** The model strictly calls defined functions (`propose_journal_entry`, `get_sales_report`, etc.),
.not raw free-form SQL

* ** .Read-Only Guardrails:** Data queries pass through a secure layer (Whitelisted Views)

.Human-in-the-Loop:** No entry is automatically posted by AI** *

.Key Management:** AES-256 encryption before storage, decrypted in memory only, never written to logs** *

Multi-Provider:** Adapter Pattern supports OpenAI, Anthropic, Gemini, Ollama, allowing seamless provider ** *
.switching without code changes

---

٨. Synchronization and Offline Mode

.Mechanism:** Simplified Event Sourcing via `sync_queue` using `operation_id` for Idempotency** *

.Conflict Strategy:** Last-Write-Wins for metadata, Append-only for financial entries** *

Frequency:** Background sync upon connection + manual sync button + WebSockets for real-time updates ** *
.between devices on the same account

Offline UI:** All buttons and saved reports function fully offline; records are cached locally and uploaded upon ** *
.reconnection

---

٩. Phased Implementation Plan

| Phase | Duration | Core Deliverables | Acceptance Criteria |

| --- | --- | --- | --- |

Phase 0:** Foundations | 2 Weeks | Monorepo, CI/CD, Core DB, Auth, Flutter skeleton. | Successful login, ** |
| .company creation, data saved locally & in the cloud

Phase 1:** Entry Engine & Operations | 3 Weeks | Posting Rule Engine, 10 operations buttons, preview screen, ** |
| .simple account statement. | Execution of 10 operations with verified journal entry balance

Phase 2:** Sync & Offline | 2 Weeks | Sync Engine, Conflict handling, 2-device testing. | Successful sync without ** |
| .data loss or duplication

Phase 3:** Inventory, HR & Assets | 2 Weeks | Inventory module, basic payroll, fixed assets. | Linking inventory ** |
| .to sales/purchases with automatic quantity updates

Phase 4:** Reports | 2 Weeks | Income Statement, Balance Sheet, Cash Flow, PDF/Excel export. | Automated ** |
| .report matches manual calculation across 3 scenarios

Phase 5:** Core AI | 3 Weeks | AI Gateway, single provider, Keys CRUD, Entry Proposal Agent. | Text description ** |
| .generates correct entry in 80%+ of 50 test scenarios

Phase 6:** Advanced AI (Voice & Analytics) | 3 Weeks | Speech-to-Text, analytical queries, RAG, multi-provider ** |
| .support. | Testing 20 voice queries + 100% analytics accuracy

Phase 7A:** Tenant RBAC | 2 Weeks | Full permissions matrix, custom roles, user management screens. | ** |
| .Testing block/allow functionality for every permission per role

Phase 7B:** Platform Admin MVP | 3 Weeks | Admin Dashboard: Overview, Merchants, Support, Billing. | ** |
| .Support agent resolves a merchant ticket without leaking another merchant's data

Phase 7C:** Billing & AI Monitoring | 2 Weeks | Subscription plans, AI token limits, auto-cutoff upon ** |
| .exceedance. | Merchant exceeding limit triggers AI shutdown with a clear notification

Phase 8:** Polish & Performance | 2 Weeks | Load testing, security audit, UX improvements, Dark/Light mode. | ** |
| .Response time < 200ms for 95% of requests, 0 critical vulnerabilities

Phase 9:** Soft Launch (Yemen) | Ongoing | Closed Beta, feedback collection, Sentry monitoring. | Satisfaction ≥ ** |
| .80%, 0 critical bugs for two consecutive weeks

Phase 10:** Official Launch (Arab World) | Ongoing | Multi-currency, tax handling, DB Sharding/K8s. | Successful ** |
| .support for multiple countries in the Arab World

---

١٠. Quality and Security Standards (Definition of Done)

:A feature is considered "Done" only if

١. ✅ .It matches the functional requirement

٢. ✅ .Unit + Integration tests pass (Coverage ≥ 80% for the accounting module, ≥ 60% for the rest)

٣. ✅ .It works offline (if related to local data)

٤. ✅ .Full support for RTL and the Arabic language

٥. ✅ .Fully documented (OpenAPI + code comments)

٦. ✅ .Code Review completed by a second party

٧. ✅ .No previously passing tests are broken (Regression)

٨. ✅ .Basic security review passed (Tenant isolation, no exposed keys, no data leaks)

٩. ✅ .Journal balancing trigger successfully fires for all templates

١٠,١ Mandatory Security Checklist

* [ ] .Local database encryption (SQLCipher)

* [ ] .TLS 1.3 for all communications

* [ ] .Rate Limiting on Auth and AI endpoints

* [ ] .RBAC enforcement on every Module/Action

* [ ] .Append-only Audit Log for every financial operation

* [ ] .tenant_id` isolation via RLS at the PostgreSQL level` [ ]

* [ ] .Automated daily backups + monthly restoration testing

---

١١. Appendices: Migrations and Seed Data

١١،١ Default Permissions Seed Data

```sql
-- Core Permissions
INSERT INTO permissions (code, module, label_ar, description_ar, is_sensitive) VALUES
('sales.create','sales','Register Sales','Register a cash or credit sale',false),
('sales.view','sales','View Sales','View the history of invoices and sales',false),
('sales.void','sales','Void Sale','Cancel a sale operation via a reversal entry',true),
('purchases.create','purchases','Register Purchases','Register a cash or credit purchase',false),
('expenses.create','expenses','Register Expense','Register a general or operational expense',false),
('expenses.approve','expenses','Approve Expense','Approve an expense exceeding the allowed limit',true),
('treasury.view_balance','treasury','View Treasury Balance','View the cash or bank balance',false),
('journal.create_manual','journal','Create Manual Entry','Create a direct manual accounting entry',true),
('journal.reverse','journal','Reversal Entry','Correction operation for a previously posted entry',true),
('inventory.view','inventory','View Inventory','View items and available quantities',false),
('inventory.adjust_stock','inventory','Stock Adjustment','Adjust inventory quantities due to counts or damage',true),
('contacts.view','contacts','View Contacts','View the list of customers and suppliers',false),
('contacts.send_statement','contacts','Send Statement','Send an account statement to a customer or supplier',false),
('reports.view_financial','reports','Financial Reports','View Income Statement and Balance Sheet',true),
('reports.export','reports','Export Reports','Export reports to PDF or Excel',false),
('ai.use_chat','ai','Use Smart Chat','Communicate with the Smart Assistant via text or voice',false),
('ai.manage_providers_keys','ai','Manage AI Providers','Add and edit AI providers and API keys',true),
('users.invite','users','Invite User','Add a new user to the company',true),
('users.manage_custom_roles','users','Manage Custom Roles','Create and edit custom roles and permissions',true),
('company.delete','company','Delete Company','Final deletion of the account and all data',true);
```

١١،٢ Default Roles (Upon New Company Creation)

sql```

Examples of inserting default roles (Executed in code post-company creation) --

Roles: Owner, Accountant, Sales Staff, Inventory Staff, Cashier, Viewer --

Each role is linked to a set of the permissions above via the role_permissions table --

```

### ١١,٣ Default Chart of Accounts Templates (Seed)

sql```

System accounts are created for every company upon registration: --

Cash / Safe (Asset) - ١٠٠٠ --

Bank (Asset) - ١١٠٠ --

Customers / Accounts Receivable (Asset) - ١٢٠٠ --

Inventory (Asset) - ١٣٠٠ --

Suppliers / Accounts Payable (Liability) - ٢٠٠٠ --

Capital (Equity) - ٣٠٠٠ --

Sales Revenue (Revenue) - ٤٠٠٠ --

General Expenses (Expense) - ٥٠٠٠ --

Salary Expense (Expense) - ٥١٠٠ --

... --

```

End of Document

This document is ready to be handed over to the development team or the AI Execution Agent to begin immediate implementation using the best architectural, security, and accounting practices.